Práctica DAO – Biblioteca (parte 1)

1. Presentación

En esta práctica vas a construir la capa de acceso a datos de una biblioteca con JDBC, organizada como un proyecto real que está distribuido en n capas: modelo, DAO, servicio y un punto de entrada para probar (que debería ser la UI).

La parte 1 (esta entrega) cubre solo consultas (SELECT); en una segunda parte (la parte 2) se añadirán el resto de operaciones CRUD, transacciones, savepoints, lotes y procedimientos almacenados sobre este mismo proyecto.

En resumen. ¿Qué tienes que hacer?:

1. Implementar los métodos genéricos `buscarPorId` y `listarTodos` en los DAO de libro, socio y préstamo (bloque A).
2. Decidir en qué DAO va cada consulta del bloque B, declararla en su interfaz e implementarla.
3. Completar los tres servicios (bloque C).

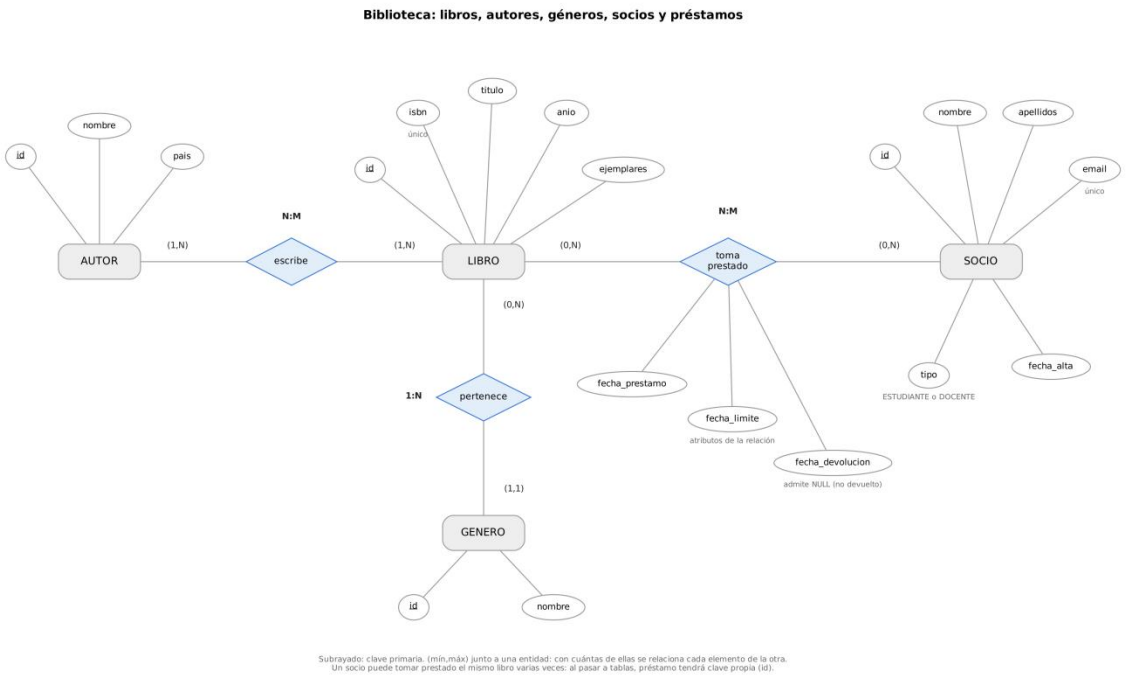
Cada ejercicio tiene un **resultado esperado**: si no te sale exactamente eso, hay un error en la consulta. Las consultas del bloque B son consultas del tipo que podría aparecer en el examen.

Recuerda

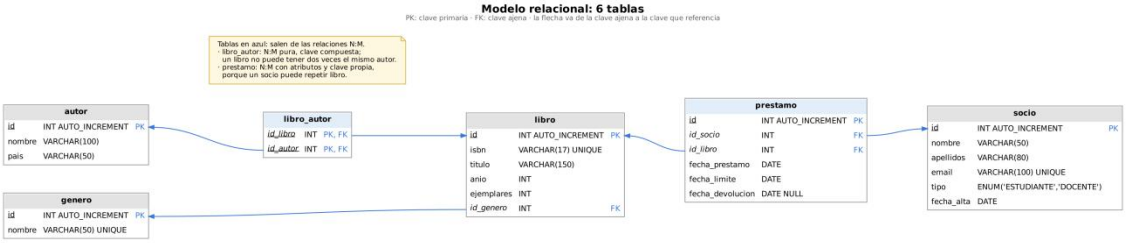
1. Cualquier valor que venga de fuera (parámetro de un método) se pasa con `PreparedStatement` y `?`, **nunca concatenado en el SQL**.
2. `PreparedStatement` y `ResultSet` siempre en **try-with-resources**.
3. Los DAO **reciben la `Connection` y no la cierran**; la abre y la cierra el servicio (ahora mismo daría igual, pero será la forma en la que más adelante podremos usar transacciones).
4. Cada fila se convierte en objeto con un método privado `mapear(ResultSet rs)`. Fuera del DAO nadie trabaja con `ResultSet`.
5. Las fechas se leen con `rs.getObject("columna", LocalDate.class)` y se pasan con `ps.setObject(n, fecha)`: así un `NULL` llega como `null`.
6. No se puede modificar la base de datos, el modelo, los records ni la interfaz `DAO<T,K>`.

2. Modelo de datos

La biblioteca tiene 6 tablas y dos relaciones N:M de distinto tipo: **libro–autor** (N:M pura) y **socio–libro** a través de los préstamos (N:M con atributos).



Al pasar a tablas, cada N:M genera una tabla. `libro_autor` tiene clave primaria compuesta por las dos claves ajenas: un libro no puede tener dos veces el mismo autor. `prestamo`, en cambio, tiene clave propia (`id`): un socio puede tomar prestado el mismo libro varias veces, así que `(id_socio, id_libro)` no sirve como clave.



Dos decisiones de diseño que afectan a las consultas:

- Un préstamo **no devuelto** tiene `fecha_devolucion` a `NULL`. Está **vencido** si no se ha devuelto y su `fecha_limite` es anterior a la fecha de referencia (la fecha cuando se consulta).
- Los ejemplares disponibles **no se guardan**: se calculan como `ejemplares` menos los préstamos sin devolver de ese libro.

Scripts

Los scripts están en la carpeta `sql/` del proyecto. Ejecútalos en MySQL Workbench en este orden; puedes repetirlos cuando quieras para volver a los datos iniciales (el primero empieza borrando la base de datos).

Script	Qué hace
`01_esquema.sql`	Crea la base de datos `biblioteca` y sus 6 tablas
`02_datos.sql`	Carga los datos de prueba

Qué se carga: 5 géneros, 11 autores, 13 libros, 15 filas en `libro_autor`, 10 socios y 38 préstamos (8 sin devolver).

Fecha de referencia. Las consultas de vencimientos reciben la fecha como parámetro en vez de usar `CURDATE()`. En todo el enunciado se usa como fecha de referencia **15/10/2026**: así los resultados esperados de las pruebas no cambian según el día en que ejecutes el programa.

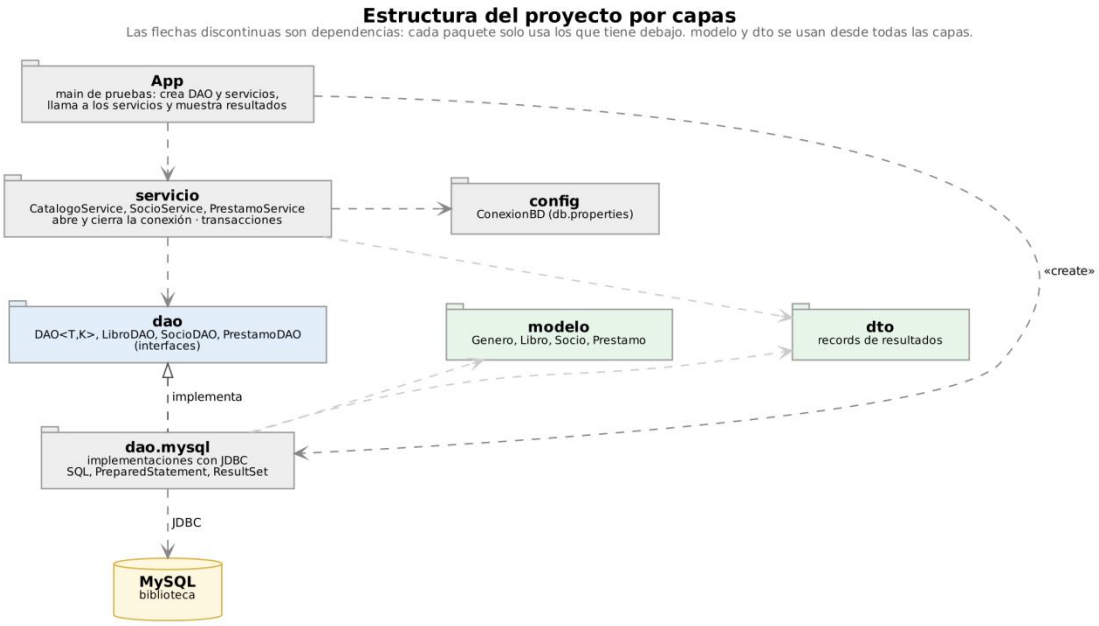
Casos especiales de los datos (son los que ponen a prueba las consultas):

- Nuno no tiene ningún préstamo
- Follas novas y Freakonomics nunca se han prestado
- Cuatro libros tienen ahora mismo todos sus ejemplares prestados
- Alba ha tomado prestado Fundación dos veces
- En Fantasía hay empate en el libro más prestado
- El préstamo de Marta vence justo el día de referencia.

3. Estructura del proyecto

```
biblioteca/
├── pom.xml
├── db.properties          ← URL, usuario y contraseña
├── db.properties.example
├── sql/                   ← 01_esquema.sql, 02_datos.sql
├── docs/
└── src/main/java/gal/dam/biblioteca/
    ├── App.java           ← punto de entrada para probar
    ├── config/            ← ConexionBD
    ├── modelo/            ← Genero, Libro, Socio, Prestamo
    ├── dto/               ← records de resultados
    ├── dao/               ← DAO<T,K> e interfaces de cada tabla
    ├── dao/mysql/         ← implementaciones con JDBC
    └── servicio/          ← CatalogoService, SocioService, PrestamoService
```

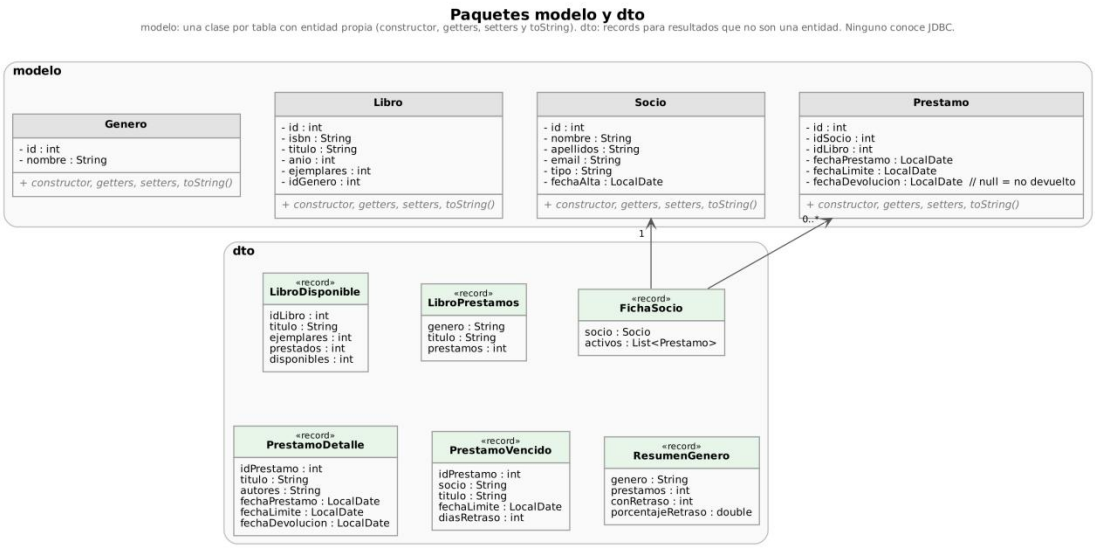
Cada paquete solo usa los que tiene debajo. `modelo` y `dto` no conocen JDBC y se usan desde todas las capas.



Qué está hecho y qué tienes que hacer

Paquete	Entregado	Por hacer
`config`	`ConexionBD` completa	—
`modelo`, `dto`	Las 4 entidades y los 6 records	—
`dao`	`DAO<T,K>`	Declarar las consultas en `LibroDAO`, `SocioDAO` y `PrestamoDAO`
`dao.mysql`	`GeneroDAOMySQL` completo, como ejemplo	`LibroDAOMySQL`, `SocioDAOMySQL`, `PrestamoDAOMySQL`
`servicio`	`listarGeneros()` y `buscarGenero()`, como ejemplo	El resto de métodos (ya están declarados)
raíz	`App` con todas las pruebas del enunciado	Puedes añadir las tuyas

Antes de empezar, mira cómo están implementados `GeneroDAOMySQL` y `CatalogoService.listarGeneros()`: todo lo que escribas debe seguir ese mismo patrón.



Las entidades guardan las claves ajenas como `int` (`idGenero`, `idSocio`, `idLibro`), no como objetos. Cuando una consulta necesita datos de varias tablas (el título del libro en un préstamo, el nombre del género...), devuelve un record de `dto`. Usa `Integer`/`Double` en vez de `int`/`double` solo si el valor puede ser `NULL`(los tipos primitivos no son nullables de por sí, los envoltorio sí).

4. Reglas de la arquitectura

La conexión la abre el servicio, no el DAO

Todos los métodos de los DAO reciben la `Connection` como primer parámetro. El DAO la usa, pero no la abre ni la cierra: por eso la `Connection` **no va** en el try-with-resources del DAO, y el `PreparedStatement` y el `ResultSet` sí.

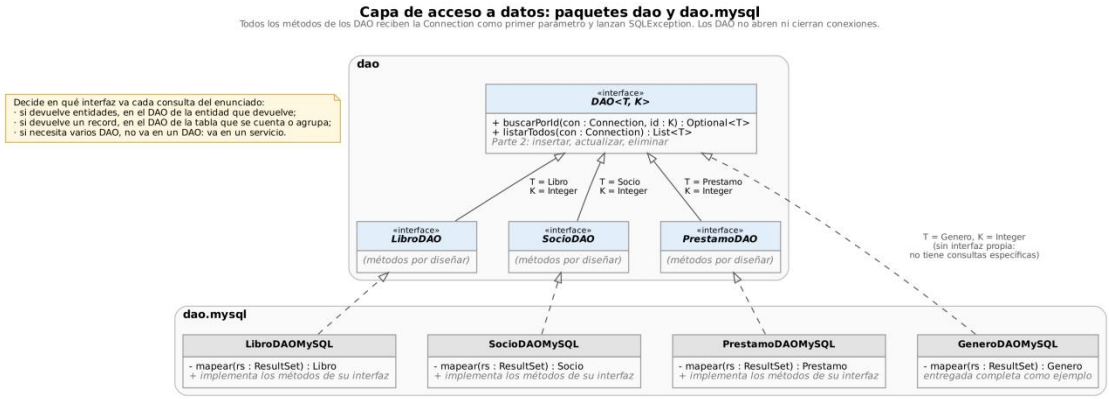
```
// En el DAO (dao.mysql): usa la conexión que recibe
public Optional<Genero> buscarPorId(Connection con, Integer id) throws SQLException {
    String sql = "SELECT id, nombre FROM genero WHERE id = ?";
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, id);
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next() ? Optional.of(mapear(rs)) : Optional.empty();
        }
    }
}

// En el servicio: abre la conexión, llama al DAO y la cierra
public Optional<Genero> buscarGenero(int id) throws SQLException {
    try (Connection con = ConexionBD.getConexion()) {
        return generoDAO.buscarPorId(con, id);
    }
}
```

¿Por qué lo hacemos así? => Un servicio puede necesitar varios DAO en la misma operación (`fichaSocio` usa dos). Con una sola conexión compartida, podemos gestionar transacciones en el servicio sin tocar los DAO. Si cada método del DAO abriese su propia conexión, eso sería imposible.

La interfaz genérica y las interfaces de cada tabla

`DAO<T,K>` define lo común a todas las tablas. Cada tabla con consultas propias tiene su interfaz, que hereda de la genérica y añade esas consultas: `LibroDAO extends DAO<Libro, Integer>`. La implementación con JDBC (`LibroDAOMySQL`) implementa la interfaz específica. Género no tiene consultas propias, así que `GeneroDAOMySQL` implementa directamente `DAO<Genero, Integer>`.



Dónde va cada consulta

Las interfaces `LibroDAO`, `SocioDAO` y `PrestamoDAO` están vacías: **decidir dónde va cada consulta del bloque B es parte del ejercicio.** Aplica estas reglas en orden:

1. Si devuelve **entidades**, va en el DAO de la entidad que devuelve, aunque filtre por otra tabla. *Libros prestados a un socio* devuelve libros: va en `LibroDAO`, no en `SocioDAO`.
2. Si devuelve un **record** que no es una entidad, va en el DAO de la tabla sobre la que se cuenta, agrupa o recorre.
3. Si necesita **resultados de varios DAO** combinados, no va en un DAO: va en un servicio, que llama a cada DAO con la misma conexión.

Tened en cuenta que el nombre del servicio no indica el DAO: `SocioService.librosLeidosDeGenero()` es una operación sobre un socio, pero devuelve libros y por tanto llama a `LibroDAO`. Un servicio puede usar cualquier DAO que reciba en su constructor.

5. Ejercicios de la capa DAO

Las firmas se dan sin el parámetro `Connection`: en el DAO, todas lo llevan delante (`buscarPorTitulo(Connection con, String texto)`). Cada ejercicio se prueba desde `App`, a través del servicio que lo expone.

Bloque A – Métodos genéricos

Implementa `buscarPorId` y `listarTodos` en `LibroDAOMySQL`, `SocioDAOMySQL` y `PrestamoDAOMySQL`, cada uno con su `mapear`.

A.1 – Libros. `listarTodos` ordenado por título.

> *Resultado esperado: 13 libros; el primero es American Gods y el último Yo, robot. `buscarPorId(4)` devuelve Fundación y `buscarPorId(99)` un `Optional` vacío.*

A.2 – Socios. `listarTodos` ordenado por apellidos.

> *Resultado esperado: 10 socios, de Eva Barreiro Mato a Nuno Varela Freire. `buscarPorId(10)` devuelve a Nuno.*

A.3 – Préstamos. `listarTodos` ordenado por fecha de préstamo y, a igualdad de fecha, por id.

> *Resultado esperado: 38 préstamos. El primero es el 2 (08/01/2026), no el 1: los ids no siguen el orden de las fechas. `buscarPorId(34)` devuelve el préstamo de Fundación a Alba, del 20/09 al 05/10, sin devolver: si te sale una fecha de devolución, estás leyendo el `NULL` mal.*

Bloque B – Consultas específicas

Para cada consulta: decide en qué interfaz va (apartado 4), declárala en ella, impleméntalo en su clase MySQL y completa el método del servicio que la usa.

B.1 – `List<Libro> buscarPorTitulo(String texto)`. Libros cuyo título **contiene** el texto, por título. El `%` va en el valor del parámetro, no en el SQL.

> *Resultado esperado: `"fundacion"` devuelve Fundación aunque se escriba sin tilde y en minúsculas. `"de"` devuelve 3: Cien años de soledad, Los desposeídos y Un mago de Terramar.*

B.2 – `List<Libro> deAutor(String nombreAutor)`. Libros de los autores cuyo nombre contiene el texto, por título. Une tres tablas a través de `libro_autor`.

> *Resultado esperado: `"gaiman"` → American Gods y Buenos presagios.*

B.3 – `List<String> autoresDe(int idLibro)`. Nombres de los autores de un libro, por orden alfabético. Devuelve `String`, no objetos: no hay clase `Autor`.

> *Resultado esperado: libro 8 → Neil Gaiman, Terry Pratchett. Libro 13 → Stephen J. Dubner, Steven D. Levitt.*

B.4 – `Optional<Socio> buscarPorEmail(String email)`.

> *Resultado esperado: `marta.rey@biblio.gal` → Marta Rey Outeiral; `nadie@biblio.gal` → `Optional` vacío.*

B.5 – `List<Prestamo> activosDeSocio(int idSocio)`. Préstamos sin devolver de un socio, por fecha de préstamo.

> *Resultado esperado: socio 1 → solo el préstamo 34. Socio 10 → lista vacía.*

B.6 – `List<Libro> prestadosASocioDeGenero(int idSocio, int idGenero)`. Libros que un socio ha tomado prestados alguna vez y que son de un género, sin repetidos, por título. El servicio lo expone como `SocioService.librosLeidosDeGenero`.

> *Resultado esperado: socio 1, género 2 (Ciencia ficción) → Fundación y Yo, robot. Si Fundación te sale dos veces, te falta algo: Alba lo tomó prestado dos veces.*

B.7 – `List<Libro> nuncaPrestados()`. Libros sin ningún préstamo, por título. Escríbela con `NOT EXISTS` y razona cómo sería con `LEFT JOIN`.

> *Resultado esperado: Follas novas y Freakonomics.*

B.8 – `List<Socio> conPrestamosVencidos(LocalDate fechaRef)`. Socios con algún préstamo sin devolver cuya fecha límite es anterior a `fechaRef`, por apellidos. Sin repetidos.

> *Resultado esperado: a 15/10/2026, Fiz Couto, Iria Mosquera, Carme Novoa y Alba Seoane. Marta no sale: su préstamo vence ese mismo día. Si sale, revisa el operador de comparación.*

B.9 – `List<PrestamoDetalle> historial(int idSocio)`. Todos los préstamos de un socio con el título del libro y sus autores en una sola cadena separada por comas (en orden alfabético), por fecha de préstamo. Pista: `GROUP_CONCAT(... ORDER BY ... SEPARATOR ', ')`.

> *Resultado esperado: socio 3 (Carme) → 5 filas. La primera es el préstamo 3, Buenos presagios, autores "Neil Gaiman, Terry Pratchett". La última es el 31, El infinito en un junco, con la fecha de devolución a `null`.*

B.10 – `List<PrestamoVencido> vencidos(LocalDate fechaRef)`. Préstamos vencidos a esa fecha con el nombre completo del socio, el título, la fecha límite y los días de retraso (`DATEDIFF`), de más a menos días de retraso.

> *Resultado esperado: a 15/10/2026, 4 filas: 30 Fiz Couto Sanmartín, Sapiens, 21 días; 31 Carme Novoa Pardo, El infinito en un junco, 14; 34 Alba Seoane Rial, Fundación, 10; 35 Iria Mosquera Lago, American Gods, 2.*

B.11 – `List<LibroDisponible> disponibilidad()`. Para **cada** libro: ejemplares, ejemplares prestados ahora (sin devolver) y disponibles. Ordenado por disponibles y después por título.

> *Resultado esperado: 13 filas. Las cuatro primeras tienen 0 disponibles: American Gods, Fundación, Los desposeídos y Que me queres, amor?. La última es Cien años de soledad, con 3. Si te salen 9 filas, has puesto la condición de "sin devolver" en el `WHERE` y no en el `ON` del `LEFT JOIN`: explica por qué desaparecen 4 libros.*

B.12 – `List<LibroPrestamos> masPrestadoPorGenero()`. Para cada género con préstamos, el libro o libros con más préstamos (si hay empate, todos), por género y título. Sin `LIMIT`: con una subconsulta correlacionada por género.

> *Resultado esperado: 5 filas: Ciencia ficción, Fundación, 6; Ensayo, Sapiens, 5; Fantasía, Buenos presagios, 4; Fantasía, Un mago de Terramar, 4; Novela, Cien años de soledad, 6. Poesía no aparece.*

B.13 – `List<Socio> lectoresDeTodoElAutor(int idAutor)`. Socios que han tomado prestados **todos** los libros de un autor, por apellidos. "No existe ningún libro de ese autor que el socio no haya tomado prestado": dos `NOT EXISTS` anidados.

> *Resultado esperado: autor 3 (Asimov) → Fiz Couto y Alba Seoane. Autor 6 (Gaiman) → Carme Novoa. Autor 9 (Rosalía de Castro) → ninguno. Breixo y Diego no salen con Asimov: solo tomaron prestado Fundación.*

B.14 – `List<Socio> lectoresSobreLaMedia()`. Socios con más préstamos que la media de préstamos por socio, contando solo los socios con algún préstamo. De más a menos préstamos y por apellidos. La media se calcula con una tabla derivada dentro del `HAVING`.

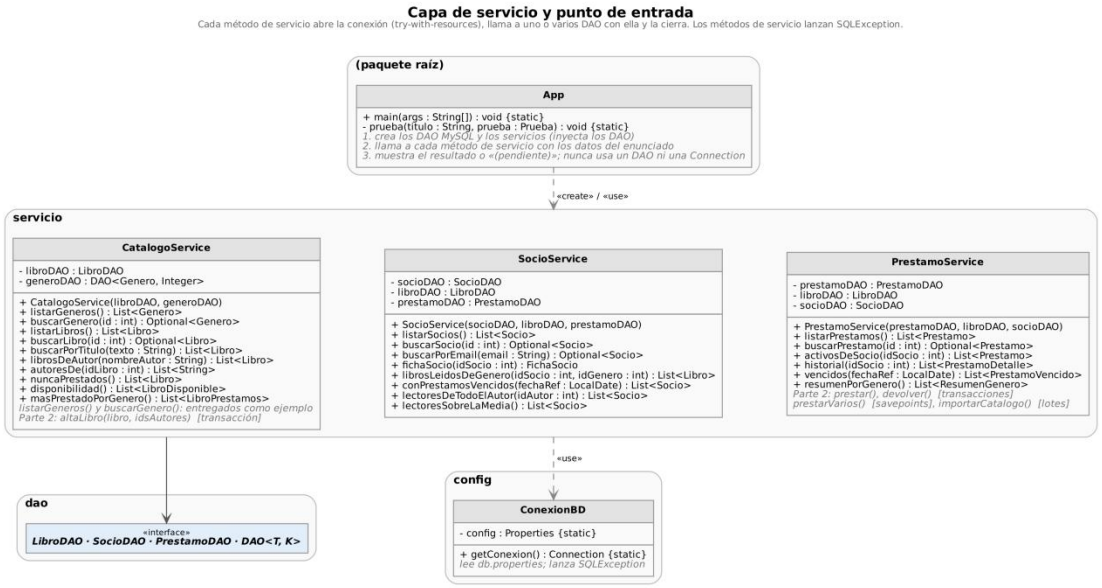
> *Resultado esperado: la media es 4,22; salen Eva Barreiro, Fiz Couto, Carme Novoa y Alba Seoane, con 5 préstamos cada uno. Pregunta: si contases también a Nuno en la media, ¿cuánto valdría y quién más saldría?*

B.15 – `List<ResumenGenero> resumenPorGenero()`. Para **cada** género: número de préstamos, cuántos se devolvieron con retraso y el porcentaje de retraso redondeado a un decimal. De más a menos préstamos y por nombre. Los préstamos sin devolver no cuentan como retraso.

> *Resultado esperado: Ciencia ficción 10 / 3 / 30.0; Fantasía 10 / 2 / 20.0; Novela 10 / 0 / 0.0; Ensayo 8 / 2 / 25.0; Poesía 0 / 0 / 0.0. Si Poesía te sale con 1 préstamo, estás usando `COUNT(*)` en vez de `COUNT(p.id)`.*

6. Capa de servicio y App

Los tres servicios ya tienen declarados sus métodos, que lanzan `UnsupportedOperationException` hasta que los completes. Cada uno recibe sus DAO por el constructor, como interfaces, y no escribe SQL.



Bloque C – Servicios

C.1 – Métodos de servicio. Completa todos los métodos de los tres servicios siguiendo el patrón de `listarGeneros()`: abrir la conexión en un try-with-resources, llamar al DAO que corresponda pasándole la conexión y devolver el resultado.

C.2 – `FichaSocio fichaSocio(int idSocio)`. Devuelve el socio con sus préstamos activos. Usa **dos DAO** (`SocioDAO` y `PrestamoDAO`) con **una sola conexión**. Si el socio no existe, lanza `NoSuchElementException("No existe el socio " + idSocio)`; pista: `orElseThrow`.

> *Resultado esperado: socio 1 → Alba Seoane Rial con un préstamo activo, el 34. Socio 99 → `NoSuchElementException: No existe el socio 99`.*

App: el punto de entrada para probar

`App` crea los DAO MySQL, se los pasa a los servicios y llama a cada método con los datos de este enunciado. Solo habla con los servicios: nunca usa un DAO ni una `Connection`.

Cada prueba muestra su título (con el número del ejercicio) y el resultado, `(pendiente)` si el método aún no está hecho, o el `SQLState` y el código de error si falla la consulta. Así puedes avanzar ejercicio a ejercicio y comparar con el resultado esperado. Puedes añadir tus propias pruebas, pero no borres las que hay.

7. Bloque D – Resto del CRUD con ejecución por lotes

Completa el CRUD de las cuatro tablas: insertar, actualizar y eliminar **una fila** y **un listado**. Los listados se hacen con **ejecución por lotes**: un solo `PreparedStatement`, `addBatch()` por cada elemento y un único `executeBatch()` al final.

Cabeceras en `DAO<T,K>`

Inserta tras el comentario «Parte 2» de la interfaz estos seis métodos.
NOTA: Al añadirlos, las cuatro clases MySQL dejarán de compilar hasta que los implementes.

```
/** Inserta la entidad (su id se ignora). Asigna al objeto el id generado y lo devuelve. */
K insertar(Connection con, T t) throws SQLException;

/** Inserta la lista en un lote. Asigna a cada objeto de la lista su id generado. */
int[] insertarLote(Connection con, List<T> lista) throws SQLException;

/** Actualiza todas las columnas de la fila con el id del objeto. false si no existe. */
boolean actualizar(Connection con, T t) throws SQLException;

/** Actualiza la lista en un lote. */
int[] actualizarLote(Connection con, List<T> lista) throws SQLException;

/** Elimina la fila con esa clave. false si no existía. */
boolean eliminar(Connection con, K id) throws SQLException;

/** Elimina en un lote las filas con esas claves. */
int[] eliminarLote(Connection con, List<K> ids) throws SQLException;
```

Los métodos de lote devuelven el array de `executeBatch()`: indica las filas afectadas por cada sentencia, en el mismo orden de la lista. Un `0` indica que esa sentencia no realizó acciones.

Cabeceras en las clases MySQL

En cada implementación, `T` y `K` se sustituyen por los tipos concretos. Por ejemplo, en `LibroDAOMySQL`:

```
@Override public Integer insertar(Connection con, Libro libro) throws SQLException
@Override public int[] insertarLote(Connection con, List<Libro> lista) throws SQLException
@Override public boolean actualizar(Connection con, Libro libro) throws SQLException
@Override public int[] actualizarLote(Connection con, List<Libro> lista) throws SQLException
@Override public boolean eliminar(Connection con, Integer id) throws SQLException
@Override public int[] eliminarLote(Connection con, List<Integer> ids) throws SQLException
```

Lo mismo en `SocioDAOMySQL` (con `Socio`) y `PrestamoDAOMySQL` (con `Prestamo`). `GeneroDAOMySQL` se entrega con los seis métodos hechos, como ejemplo.

Cabeceras en los servicios

Cada método abre la conexión, llama al método del DAO correspondiente y devuelve su resultado. El nombre en singular trabaja con una fila; el plural, con un lote.

```
// CatalogoService
public int insertarGenero(Genero genero) throws SQLException
public int[] insertarGeneros(List<Genero> generos) throws SQLException
public boolean actualizarGenero(Genero genero) throws SQLException
public int[] actualizarGeneros(List<Genero> generos) throws SQLException
public boolean eliminarGenero(int id) throws SQLException
public int[] eliminarGeneros(List<Integer> ids) throws SQLException


public int insertarLibro(Libro libro) throws SQLException
public int[] insertarLibros(List<Libro> libros, boolean transaccional) throws SQLException
public boolean actualizarLibro(Libro libro) throws SQLException
public int[] actualizarLibros(List<Libro> libros) throws SQLException
public boolean eliminarLibro(int id) throws SQLException
public int[] eliminarLibros(List<Integer> ids) throws SQLException


// SocioService
public int insertarSocio(Socio socio) throws SQLException
public int[] insertarSocios(List<Socio> socios) throws SQLException
public boolean actualizarSocio(Socio socio) throws SQLException
public int[] actualizarSocios(List<Socio> socios) throws SQLException
public boolean eliminarSocio(int id) throws SQLException
public int[] eliminarSocios(List<Integer> ids) throws SQLException


// PrestamoService
public int insertarPrestamo(Prestamo prestamo) throws SQLException
public int[] insertarPrestamos(List<Prestamo> prestamos) throws SQLException
public boolean actualizarPrestamo(Prestamo prestamo) throws SQLException
public int[] actualizarPrestamos(List<Prestamo> prestamos) throws SQLException
public boolean eliminarPrestamo(int id) throws SQLException
public int[] eliminarPrestamos(List<Integer> ids) throws SQLException
```

A tener en cuenta en el bloque D

1. `insertar` prepara la sentencia con `Statement.RETURN_GENERATED_KEYS`, lee el id con `getGeneratedKeys()` y lo asigna al objeto con `setId`.
2. `insertarLote` hace lo mismo después de `executeBatch()`: `getGeneratedKeys()` devuelve una clave por fila, en el orden de la lista.
3. `actualizar` y `eliminar` devuelven `executeUpdate() == 1`.
4. Un préstamo sin devolver se inserta con `fecha_devolucion` a `NULL`: `ps.setObject(n, null)` o `ps.setNull(n, Types.DATE)`.
5. Sin transacciones, salvo en D.7: cada sentencia se confirma al ejecutarse (*autocommit*).

Pruebas en App

Las pruebas están en `probarCrudGeneros`, `probarCrudLibros`, `probarCrudSocios` y `probarCrudPrestamos`, comentadas. Descomenta cada una cuando tengas los métodos de esa tabla.

Cada una inserta, modifica y borra sus propias filas, así que la base de datos acaba como estaba. Cada prueba lleva su resultado esperado en un comentario.

Los ids esperados son los de la primera ejecución después de cargar los scripts 01 y 02: Teatro 6, Cómic 7 y Biografía 8; los libros nuevos 14, 15 y 16; los socios 11, 12 y 13, y los préstamos 39, 40 y 41. En ejecuciones posteriores, `AUTO_INCREMENT` sigue contando.

Prueba	**Error esperado**
Insertar género, ISBN o email repetido	SQLState 23000, código 1062
Insertar libro con género 99 o préstamo del socio 99	SQLState 23000, código 1452
Eliminar género 1, libro 4 o socio 1	SQLState 23000, código 1451
Insertar préstamo con fecha límite anterior a la del préstamo	SQLState HY000, código 3819
Eliminar en lote los libros nuevo, 4 y nuevo	`BatchUpdateException`, código 1451; resultado por sentencia `[1, -3, 1]`

D.7 – Inserción en lote con y sin transacción. `CatalogoService.insertarLibros` recibe un segundo parámetro, `boolean transaccional` (solo tienes que tocar el servicio):

- `false`: cada sentencia del lote se confirma al ejecutarse (*autocommit*), como en el resto de lotes.
- `true`: el lote es atómico. O se insertan todos los libros, o ninguno: si falla una sentencia, se deshace todo y se relanza la excepción.

La prueba inserta dos veces un lote de tres libros cuyo segundo ISBN está repetido: primero con `false` y después con `true`. Los libros de prueba tienen "Prueba D.7" en el título.

> *Resultado esperado:* en los dos casos el lote lanza una `BatchUpdateException`, código 1062, con `[1, -3, 1]`. Con `false`, la búsqueda de "Prueba D.7" devuelve 2 libros (A y C) y la limpieza los borra con `[1, 1]`. Con `true`, la búsqueda devuelve una lista vacía. Al final quedan los 13 libros iniciales.

7. Entrega y evaluación

Se entrega el proyecto completo comprimido en un `.zip` (mejor sin la carpeta `target`).

Criterio	Peso
Bloque A: métodos genéricos y `mapear` correctos (incluido el `NULL` de las fechas)	10 %
Bloque B: resultados exactos de las 15 consultas	35 %
Ubicación de cada consulta en el DAO correcto e interfaces bien declaradas	10 %
Bloque C: servicios, `fichaSocio` con una sola conexión y respuesta a C.3	10 %
Bloque D: CRUD de 1 fila y en lote en los tres DAO y sus servicios (ids generados asignados, `int[]` de `executeBatch`, errores de integridad)	20 %
D.7: `insertarLibros` con y sin transacción (`commit`/`rollback` y relanzar la excepción) y respuesta a la pregunta	5 %
Normas: `PreparedStatement`, try-with-resources, la `Connection` solo la cierra el servicio	10 %
Total	**100 %**

Una consulta que concatena un valor en el SQL, o un DAO que abre o cierra la conexión, puede llevar penalización aunque el resultado sea correcto.